

Создание устройства в /dev при загрузке модуля ядра

Владислав Богданов

2026-07-09

Оглавление

1. Файл устройства.....	1
2. Базовый модуль	1
3. Современный - автоматическое создание в С (рекомендуемый).	3
4. Старый (в bash)	4

1. Файл устройства

Что такое файл устройства на самом деле

Файл устройства в `/dev` — это интерфейс или точка входа в драйвер ядра. Это как "дверь", через которую `userspace`-программы обращаются к драйверу. Когда ты открываешь `/dev/radio` и читаешь из него, происходит следующее:

```
Userspace (твоя программа)
↓
open("/dev/radio")
↓
Ядро находит major/minor номер
↓
Ядро находит драйвер по major номеру
↓
Вызывается функция radio_open() из твоего драйвера
↓
read(fd, buf, size)
↓
Вызывается функция radio_read() из твоего драйвера
↓
Драйвер возвращает данные (или ошибку)
```



Сам файл `/dev/radio` не содержит данных — это просто "заглушка" в файловой системе, которая говорит ядру: "когда кто-то обратится ко мне, вызови функции из такого-то драйвера".

2. Базовый модуль

Просто минимально запускаемый модуль:

```
// radio.h
#ifndef _RADIO_H_
#define _RADIO_H_

#include <linux/module.h>

#define DEV_NAME "radio"

#endif /* _RADIO_H_ */

// radio_main.c
#include "radio.h"
```

```

MODULE_LICENSE("GPL");
MODULE_AUTHOR("user");

static struct file_operations fops;
static int radio_major;

int init_module(void)
{
    memset(&fops, 0, sizeof(fops));

    // регистрация устройства
    // первый параметр это желаемый major
    // 0 - говорит что бы система назначила номер автоматически
    // другой номер - система будет пытаться назначить выбранный номер
    radio_major = register_chrdev(0, DEV_NAME, &fops);
    if(radio_major < 0)
    {
        printk(KERN_ALERT DEV_NAME " was not registered\n");
        return radio_major;
    }
    printk(KERN_INFO DEV_NAME " registered, major number = %d\n", radio_major);

    return 0;
}

void cleanup_module(void)
{
    unregister_chrdev(radio_major, DEV_NAME);
    printk(KERN_ALERT DEV_NAME " exited\n");
}

```

При загрузке данного модуля функция *register_chrdev()* только регистрирует major-номер в */proc/devices*, но не создает файл устройства в */dev*.

Есть два способа решить задачу:

- современный (в C)
- старый (в bash)



Если у тебя в системе не запущен udev (например, какой-нибудь голый embedded-линукс без systemd/eudev), то *device_create* файл в */dev* не создаст — тогда придётся либо запускать udev, либо использовать *mknod* в стартовом скрипте.

Проверить, запущен ли демон *udev*

```

# Для систем с systemd (большинство современных дистрибутивов)
systemctl status systemd-udev

```

```
# Или более старый способ – через ps
ps aux | grep udevd
```

3. Современный - автоматическое создание в С (рекомендуемый)

Начиная с ядер 2.6, используется механизм `class_create` + `device_create`. При загрузке модуля ядро посылает `uevent`, а `udev` (демон в `userspace`) автоматически создаёт файл в `/dev`. Вот как должен выглядеть твой код:

```
#include "radio.h"
#include <linux/fs.h>
#include <linux/device.h>

MODULE_LICENSE("GPL");
MODULE_AUTHOR("user");

static struct file_operations fops;
static int radio_major;
static struct class *radio_class;
static struct device *radio_device;

int init_module(void)
{
    memset(&fops, 0, sizeof(fops));

    // 1. Регистрируем major-номер
    radio_major = register_chrdev(0, DEV_NAME, &fops);
    if(radio_major < 0)
    {
        printk(KERN_ALERT DEV_NAME " was not registered\n");
        return radio_major;
    }
    printk(KERN_INFO DEV_NAME " registered, major number = %d\n", radio_major);

    // 2. Создаём класс устройства
    //ядра < 6.4
    //radio_class = class_create(THIS_MODULE, "radio_class");
    // В ядрах 6.4+ первый параметр THIS_MODULE УБРАН!
    radio_class = class_create("radio_class");
    if(IS_ERR(radio_class)) {
        unregister_chrdev(radio_major, DEV_NAME);
        return PTR_ERR(radio_class);
    }

    // 3. Создаём устройство – триггерит udev, создаёт /dev/radio
    radio_device = device_create(radio_class, NULL,
        MKDEV(radio_major, 0),
```

```

        NULL, DEV_NAME);

    if(IS_ERR(radio_device)){
        class_destroy(radio_class);
        unregister_chrdev(radio_major, DEV_NAME);
        return PTR_ERR(radio_device);
    }

    printk(KERN_INFO DEV_NAME " registered, major = %d, /dev/%s created\n",
           radio_major, DEV_NAME);
    return 0;
}

void cleanup_module(void)
{
    // Уничтожаем в обратном порядке
    device_destroy(radio_class, MKDEV(radio_major, 0));
    class_destroy(radio_class);
    unregister_chrdev(radio_major, DEV_NAME);
    printk(KERN_ALERT DEV_NAME " exited\n");
}

```

Что здесь происходит по шагам:

- `register_chrdev()` — резервирует `major`-номер.
- `class_create()` — создаёт «категорию» устройств (видна в `/sys/class/`).
- `device_create()` — регистрирует конкретное устройство. Ядро рассылает `uevent`, и `udev` автоматически создаёт `/dev/radio`.



Обрати внимание на порядок очистки: `device_destroy` → `class_destroy` → `unregister_chrdev`. Если перепутаешь, будет Oops.

4. Старый (в `bash`)

Если ты оставишь свой исходный код как есть, то после `insmod` в `/proc/devices` запись появится, а в `/dev` — нет. Тогда файл создаётся вручную:

```

# Смотрим, какой major-номер выделило ядро
cat /proc/devices | grep radio

# Допустим, выдало 240. Создаём устройство:
sudo mknod /dev/radio c 240 0
#          ллллл    л лл л
#          имя      | | minor
#                  тип (с = character)
#                  major

```

```
# Задаём права, иначе только root сможет открыть  
sudo chmod 666 /dev/radio
```

При выгрузке модуля

```
sudo rm /dev/radio
```

Этот способ считается устаревшим — его используют только для отладки или на системах без udev (например, в initramfs на ранних этапах загрузки).